

Fused BVH to Ray Trace Level of Detail Meshes

Paritosh Kulkarni
Advanced Micro Devices, Inc.
Canada
Paritosh.Kulkarni@amd.com

Sho Ikeda
Advanced Micro Devices, Inc.
Japan
Sho.Ikeda@amd.com

Takahiro Harada
Advanced Micro Devices, Inc.
USA
Takahiro.Harada@amd.com

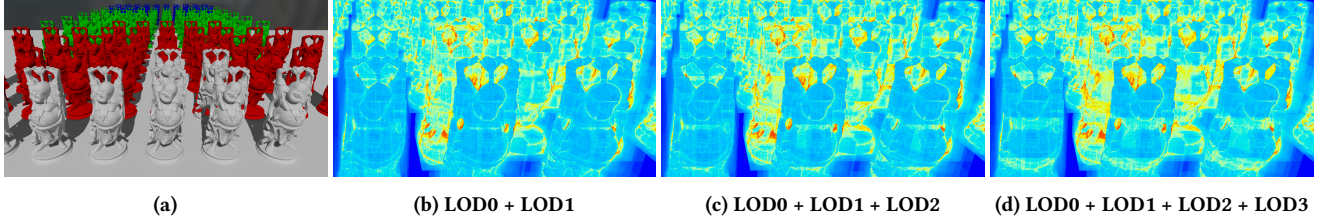


Figure 1: (a) The path traced image rendered with the fused BVH method with 4 LOD levels. Distance based LOD with diffuse and shadow rays. (b), (c), (d) Zoomed view of (a). Shows a heat map of the number of nodes visited per pixel during ray cast for primary rays with the fused BVH. We set 1280 x 720 resolution and $T=256$. As we can see with the fused BVH the number of nodes visited are increased as we add LODs.

ACM Reference Format:

Paritosh Kulkarni, Sho Ikeda, and Takahiro Harada. 2022. Fused BVH to Ray Trace Level of Detail Meshes. In *SIGGRAPH Asia 2022 Posters (SA '22 Posters)*, December 06-09, 2022. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3550082.3564211>

1 INTRODUCTION

The level of detail (LOD) is one of the optimizations tried to reduce the cost of ray casting. Current approaches like [Lee et al. 2019] need an extension to the ray tracing pipeline. While LOD approach [Ikeda et al. 2022] uses existing BVH to simulate LOD. [Ikeda et al. 2022] improves the ray casting performance but introduces some darkening bias.

The paper presents different strategies for fusing Bounding Volume Hierarchies (BVHs) of varying LOD meshes. Our method makes it possible to traverse BVHs of varying LODs in a cache-friendly way in path tracing. Considering BVH of the finest level (we call it the Base LOD BVH), the method defines a strategy to fuse subtrees from BVH of other LODs to the Base LOD BVH. Our method proposes a heuristic to find such subtrees in non-Base LOD BVHs and an algorithm to find fusion or insertion points for these subtrees in the Base LOD BVH without much increasing the overall volume of the Base LOD BVH and the number of nodes traversed as a result of this fusing. We explain the method using two LOD levels 0 and 1 in this poster. But the method is applicable to many LODs.

2 FUSED BVH FOR LOD MESHES

2.1 Finding a Mesh Cut

We start by finding subtrees forming a cut in LOD 1 BVH at decent depth. For this purpose, we introduce a threshold T as a number of child nodes. We start traversing LOD 1 BVH until we reach a node with the number of child nodes less than or equal to T . We add such a node to a mesh cut and continue traversal till we have visited all the nodes as shown in Fig 2a. If we want a finer mesh cut, we can set $T=1$. This T will result in a mesh cut with all the triangle primitives.

2.2 Fusing Mesh Cut in the Base LOD BVH

We go through all nodes in a mesh cut and insert them at an appropriate internal nodes in the Base LOD BVH by traversing the Base LOD BVH from the root node. At each node in the Base LOD BVH, if the node overlaps a node in a mesh cut we move down the Base LOD BVH. We also check the termination criteria at each node for which we use the surface area of an AABB of the node. We terminate at the first Base LOD BVH node for which a node in the mesh cut when merged is increasing the surface area. We allow inserting only a single node in a mesh cut to a single node in the Base LOD BVH. This simplifies the data structure and the traversal logic. However, the logic can report the same node from the Base LOD BVH as the insertion point for multiple nodes in a mesh cut. This results in a conflict of insertion. When a conflict happens, we descend the conflicted node in the Base LOD BVH choosing the child that has the maximum overlapped surface area with a node in a mesh cut and report it as the insertion point. After all nodes in a mesh cut are inserted, we set a flag for a terminator of a LOD. This is necessary to avoid unnecessary traversal as shown in Fig 2b. Think of a case where we traverse the Base LOD BVH to find an intersection for LOD 1 BVH. Then after testing a LOD 1 fused node, we do need to descend the Base LOD BVH if there is another node containing a LOD 1 fused node in the descendants. However, if

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA '22 Posters, December 06-09, 2022, Daegu, Republic of Korea

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9462-8/22/12.

<https://doi.org/10.1145/3550082.3564211>

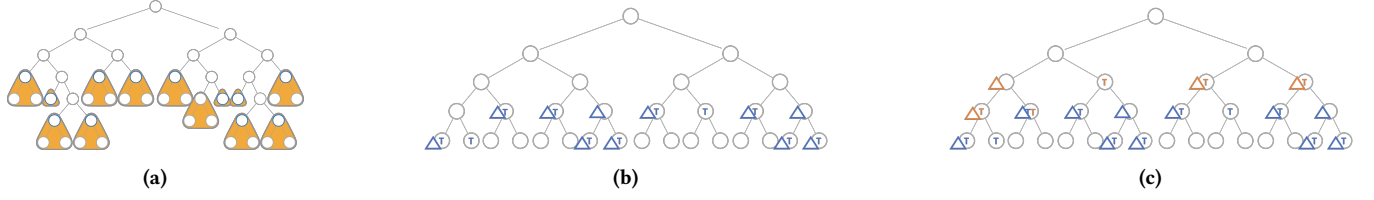


Figure 2: (a) A mesh cut formed by subtrees from LOD 1 BVH with a specified threshold. (b) An illustration of the fusion of subtrees from a single LOD to the Base LOD BVH. Note that the inserted subtrees in blue do not form a cut of the tree. The terminal nodes with the label "T" in this figure form a cut. (c) An illustration of multiple LODs fused in the Base LOD BVH.

there is no node containing a LOD 1 fused node in the descendants, we do not need to descend the Base LOD BVH. Therefore, we tag the very last fused node with LOD 1 as a terminator. This fused node can be found by a bottom-up traversal. However, there is no guarantee that nodes with LOD 1 primitives form a cut of the BVH as shown in Fig 2c. as the fused node insertion is arbitrary. When there is a part of the tree where no fused node exists for LOD 1, we traverse the Base LOD BVH to the leaf node which is not necessary if we are only interested in intersecting against LOD 1 primitives. Therefore, we also need to insert a terminator flag to a node in the Base LOD BVH where we can say there is no need to descend the tree anymore. The node we need to flag can be found by another traversal of the Base LOD BVH. The condition is if a node does not have a descendant with a terminal flag and the sibling of the node has a descendant with a terminal flag, the node needs to be flagged as a terminal. The last thing we need to do is refit the AABBs in the Base LOD BVH. We calculate inefficiency cost using a cost model for the nodes in a mesh cut and use it to define the order of fusion. The details and the effects of the cost model are provided in the supplement document.

2.3 Traversal of Fused BVH

The traversal of a fused BVH is similar to a traversal of a regular BVH but with an additional conditional execution at each node in the Base LOD BVH. When we visit a node, we check the additional data as described in the previous section and perform an intersection if the node has the level we are interested in. From this point on we will continue traversing the LOD subtree that is fused at this node. After that, we check the terminal flag of the level in the node. If the node is a terminal of the level, we skip the traversal of its children. Otherwise, we execute the traversal as usual.

3 RESULTS

The process of fusing LOD BVH does increase the overall volume of the Base LOD BVH but this increase is very well compensated by the reduction in the memory overhead. To quantify the memory overhead reduction we used Buddha, Dragon, and Erato models. The result of the fusing is the BVH with a very compact memory layout. The LODs other than the Base BVH are reduced in memory as seen from Table 1. We achieve 60.73%, 60.74%, and 60.73% reduction in the memory overhead for Buddha, Dragon, and Erato models, respectively. The resulting reduction in the memory overhead will eventually reduce the memory leading to better performance in ray casting.

Table 1: LODs and size of the BVHs in Kb for the Baseline vs the Fused BVH. Buddha model LOD 0, 1, 2, 3 with 200K, 80K, 10K, 3K primitives respectively. Dragon and Erato models LOD 0, 1, 2, 3 with 400K, 200K, 80K, 10K primitives respectively. LOD 0 BVH or the Base BVH is not affected by fusion hence its sizes are not provided.

LOD	Buddha Base-line(kb)	Buddha Fused T=256	Dragon Base-line	Dragon Fused T=256	Erato Base-line	Erato Fused T=256
1	11135.29	6927.6	25651.13	15957.82	24815.80	15439.39
2	1391.68	865.74	10260.41	6383.61	10031.80	6240.60
3	389.44	242.27	2565.05	1595.64	2639.80	1642.43

Table 2: Measurement of nodes/pixel visited with multiple LODs with Buddha scene.

number of nodes visited per pixel nodes/pixel				
LOD levels	Baseline	Fused BVH T=128	Fused BVH T=256	Fused BVH T=512
0+1	480.18	584.82	559.05	552.13
0+1+2	479.96	603.02	570.16	559.05
0+1+2+3	479.89	617.90	579.75	560.99

To quantify the number of nodes visited per pixel we prepared a scene with 100 instances of the Buddha model, 4 LODs fused in the Base LOD BVH with different values of T and compare it with [Lee et al. 2019] method (we consider it baseline) using our in-house path tracer. As we fuse more LODs or use smaller values for T the number of subtrees to be fused increases resulting in more collisions and causing the subtrees to be fused at higher depth. This will increase the overall volume of the Base BVH; hence, the number of nodes visited per pixel is increased from 9% to 13% as we can see in Fig 1b, 1c, 1d and Table 2. We observed with $T = 256$ we got an optimal trade-off between memory compaction and an increase in the number of nodes visited.

REFERENCES

- Sho Ikeda, Paritosh Kulkarni, and Takahiro Harada. 2022. Multi-Resolution Geometric Representation using Bounding Volume Hierarchy for Ray Tracing. *GPUOpen technical report 22-02-f322* (2022).
- Won-Jong Lee, Gabor Lektor, and Karthik Vaidyanathan. 2019. Flexible Ray Traversal with an Extended Programming Model. In *SIGGRAPH Asia 2019 Technical Briefs*. 17–20.